

AI术语解析-框架篇

 AI Technology Series

从 RAG 到智能体协作



检索增强生成 (RAG)

突破大模型知识边界，实现精准的外部数据融合与生成



智能体协作 (Agent Collaboration)

构建多智能体分工网络，解决复杂任务的自动化与协同

01 RAG与Embedding：企业级语义检索新范式

从传统关键词匹配迈向深度语义理解，重构企业知识获取与利用的核心逻辑



Embedding: 赋予机器“语义之眼”

👁️ 核心定义：从文本到向量

将非结构化文本转化为低维稠密向量，利用向量空间距离量化语义相似度，是实现计算机“语义理解”的关键技术。

🔧 核心技术特征

- ✓ 语义相似性量化：距离即意义
- ✓ 跨格式支持：兼容代码、文档等多模态
- ✓ 高效计算：低维向量，计算成本可控
- ✓ 生态兼容：与大语言模型(LLM)无缝集成



📈 开发效能提升

40%+

代码复用率提升

60%

问题解决效率提升

RAG：检索增强生成，连接通用与私有知识

💡 核心定义与价值

结合Embedding检索与LLM生成，解决传统大模型“知识过时”和“不懂企业私有数据”的痛点，实现精准生成。

👤 通俗理解

为LLM配备“企业私有知识检索器”。生成内容前先查阅内部文档，确保输出既懂通用知识，又贴合企业实际业务。

🔄 核心工作流程



检索

- Step 1:需求输入 & Embedding检索 (私有知识库)
- Step 2:检索结果拼接 (构建上下文)
- Step 3:LLM生成 (基于上下文输出精准内容)

生成

RAG开发场景：企业级代码问答与老旧代码重构



企业级代码问答

新员工提问，RAG检索接口文档和历史代码，生成完整调用代码，问题解决时间从1小时缩短至5分钟。



老旧代码重构建议

输入老旧代码，RAG检索重构案例和技术规范，生成针对性重构建议和代码，重构效率提升50%。

02 AtoA智能体协作：AI开发的团队化革命



从“全能选手”到“专业团队”

打破单一模型的局限性，将复杂任务拆解为多个专精领域的智能体，实现分工明确的高效协作。



任务拆解与智能协同

利用AtoA通信协议，智能体间可自动分配角色、传递信息并验证结果，构建自动化的工作流闭环。



开发效能的指数级飞跃

通过并行计算与专业化分工，大幅降低开发门槛，让AI应用的构建速度与质量同时得到质的提升。

AtoA：智能体的专业分工与高效协同

核心定义

多Agent协同工作模式，核心在于“专业分工、协同高效”，通过各司其职实现目标。

通俗理解：从个人到团队

告别单一Agent的“全能型开发者”模式，构建由需求、代码、测试Agent组成的虚拟“开发团队”。

核心特征

职责专业化、任务自动拆解、多Agent通信机制、闭环迭代优化。



AtoA核心逻辑：拆解→分配→同步→修正→闭环



任务拆解

主Agent将复杂任务拆解为可执行的子任务，明确执行路径。



Agent分配

根据子任务类型，精准分配给对应专业Agent执行。



结果同步

各Agent完成任务后，实时同步成果给关联Agent。



问题修正

测试Agent发现问题，即时反馈给对应Agent进行优化。



闭环交付

所有子任务通过测试，整合成果完成最终交付。

AtoA开发价值：质量、效率、复杂任务全覆盖



任务质量提升

专业Agent聚焦单一环节，通过精细化分工，将bug率从15%大幅降低至3%以下。



开发效率翻倍

多Agent并行工作模式，使复杂模块开发周期从10天缩短至3天，整体效率提升70%以上。



复杂任务覆盖

成功覆盖电商系统、CRM、ERP等大型项目模块开发，适配率高达95%以上。

AtoA应用案例：CRM客户管理模块开发

智能体协作分工

- 需求分析 Agent：输出精准需求文档
- 后端/前端 Agent：并行同步进行代码开发
- 测试 Agent：开发中同步介入，实时反馈

核心效能亮点

- 极致提效：7小时 vs 传统5天，效率提升17倍
- 全自动化：端到端无人工干预，质量高度可控
- 架构优势：模块化并行设计，系统可扩展性强



03 LangChain与LangGraph：AI应用开发的双引擎



LangChain：兵工厂 (Infrastructure)

作为AI应用开发的基础设施，LangChain提供了丰富的组件库和工具链，极大地降低了连接大语言模型与外部数据的门槛，是构建复杂Agent的基石。



LangGraph：指挥系统 (Orchestration)

专注于多Agent协作与复杂工作流的编排。它定义了状态管理和图结构，让Agent具备记忆、反思和多轮调用能力，实现更高级的逻辑闭环。

LangChain: AI应用开发的“预制构件厂”

核心定义：整合与标准化

AI应用开发框架，封装LLM调用、数据检索、工具集成等基础功能，实现标准化。

核心价值：降本增效

降低开发门槛，实现模块复用；兼容多模型工具，支持快速原型验证。

通俗理解：搭积木式开发

提供现成的“地基”与“墙体”模块，开发者可快速组合搭建企业级AI应用。



LangChain开发示例：30行代码搭建代码问答系统

核心逻辑架构

通过 LangChain 组合四大核心模块：LLM 大模型调用、Embedding 向量化、向量存储管理以及 RAG 检索增强生成，构建完整的问答闭环。

开发效率飞跃

无需手动编写底层通信与检索逻辑，直接复用成熟模块，开发周期大幅缩短，整体效率提升**80%**。

核心代码实现 (Python)

```
# 1. 初始化组件
llm = OpenAI(model_name="gpt-4")
vector_db =
Chroma(embedding_function=OpenAIEmbeddings())
# 2. 构建RAG检索链
qa_chain = RetrievalQA.from_chain_type(
    llm=llm, retriever=vector_db.as_retriever()
)
# 3. 调用问答系统
result = qa_chain({"query": "如何调用内部订单接口? "})
```

LangGraph: 复杂AI流程的“智能编排师”

核心定义：有状态的循环式工作流

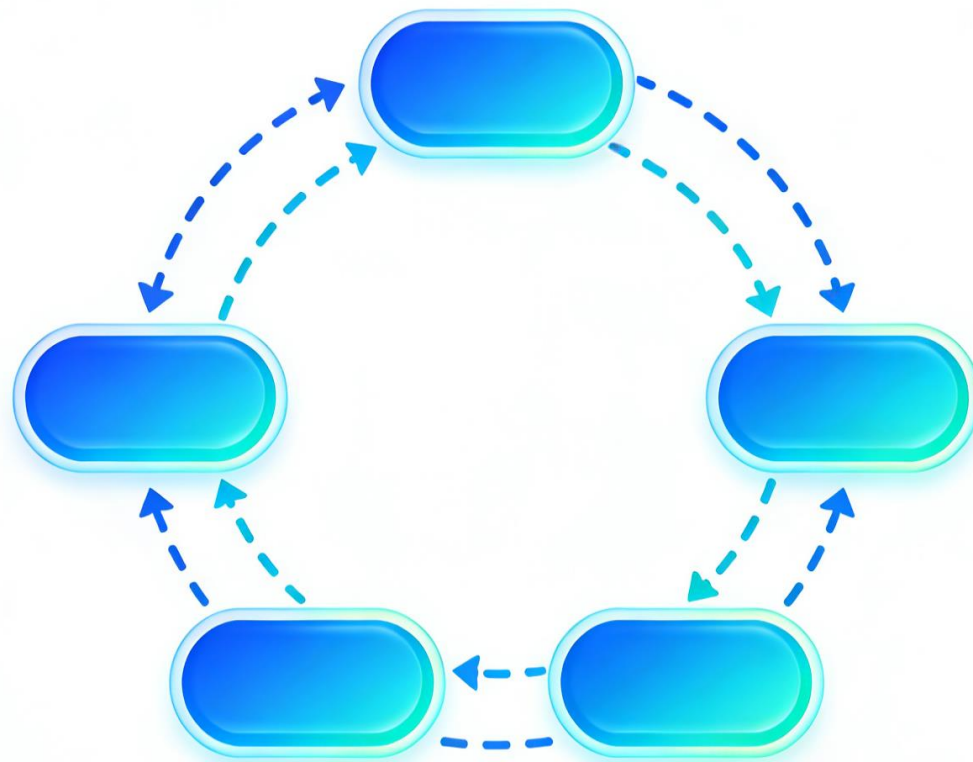
基于LangChain构建，专注解决复杂流程编排问题，支持记忆与多轮循环交互。

核心价值：可视化的节点与边

将工作流抽象为节点和边，擅长处理复杂的条件分支与循环逻辑，实现可视化编排。

与LangChain的定位差异

LangChain解决“模块组装”问题，而LangGraph专注于“流程管理与状态流转”。



LangGraph开发实操：代码生成→检查→修复自动化



全自动化流程闭环

- 生成Java工具类代码
- SonarQube代码质量检查
- 自动修复检测到的高危漏洞
- 循环迭代直至无漏洞



智能条件流转控制

- 基于条件边(Conditional Edge)决策
- 判定逻辑：存在高危漏洞？
- 分支1：有漏洞 → 跳转修复节点
- 分支2：无漏洞 → 流程结束



开发效率质的飞跃

- 实现复杂流程的全自动迭代
- 完全无需人工介入代码修复
- 确保交付代码的高安全性
- 释放开发者精力专注于设计

感谢聆听

Q & A 互动交流

